M09: SSH Keys and Security: Lock the Door

> "Bots are hammering your server with 10,000 password attempts per day. Switch to SSH keys and lock the door."

Quick Check

Show of hands:

- Who has heard "use SSH keys" but still uses passwords?
- Who has wondered how SSH keys actually work?
- Who has been locked out after disabling passwords?

> Today you'll learn the authentication method that makes brute-force attacks impossible — the same method used by every major cloud provider and tech company.

What You'll Walk Away With

By the end of this mission, you'll be able to:

- **Understand the real attack statistics** that make passwords dangerous
- **Generate SSH key pairs** with proper algorithms
- **Install keys on servers** using ssh-copy-id
- **Disable password authentication** safely
- **Configure SSH shortcuts** with ~/.ssh/config
- **Troubleshoot common SSH errors**

Commands you'll learn: `ssh-keygen`, `ssh-copy-id`, `ssh`, `hostname`, `hostnamectl`

Why Password Authentication Is Risky

Real attack statistics:

Fact	Number
Brute force attempts per server per day	10,000+
Time to crack a weak password	Minutes to hours
Percentage of breaches involving stolen credentials	~50%
Default SSH port scans per hour	100+

Password Attack Vectors

Risk	Reality
Brute force	Bots try thousands per hour
Password reuse	Users pick weak or repeated passwords
Credential theft	Passwords can be phished or leaked
Shared access	Hard to revoke one person's password

> "In cloud, password SSH authentication is disabled by default. There's a reason."

How SSH Keys Work

```
You have:                    Server has:
+----------------+           +------------------+
| Private Key    |           | Public Key       |
| (id_ed25519)   |  ------>  | (authorized_keys)|
| NEVER SHARE!   |   proves  | Can share freely |
+----------------+  identity +------------------+
```

The handshake:

1. You connect: `ssh anas@server`
2. Server sends a random challenge
3. Your private key signs the challenge
4. Server verifies with your public key
5. If match → access granted. No password needed.

Understanding Public/Private Keys

Think of it like padlocks:

Public key

A padlock you can give to anyone

Private key

The only key that opens YOUR padlocks

How it works:

- You send padlocks (public keys) to servers
- Only your key (private key) opens them
- Anyone can have the padlock
- Only you have the key

> This is **asymmetric cryptography** — the foundation of modern security.

ssh-keygen — Generating SSH Keys

What it does: Creates a new SSH key pair (public + private).

Run this on YOUR laptop, NOT on the server!

```
# Generate Ed25519 key (recommended)
ssh-keygen -t ed25519 -C "anas@tuwaiq"

# Or RSA for wider compatibility
ssh-keygen -t rsa -b 4096 -C "anas@tuwaiq"
```

Key type comparison:

Key Type	Security	Speed	Compatibility
`ed25519`	Excellent	Fast	Modern systems
`rsa 4096`	Excellent	Slower	Everything

What ssh-keygen Asks

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/anas/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase): ********
Your identification has been saved in /home/anas/.ssh/id_ed25519
Your public key has been saved in /home/anas/.ssh/id_ed25519.pub
```

Important choices:

- **File location:** Default is fine
- **Passphrase:** YES, add one (second layer of protection)

> **Pro tip:** A passphrase protects your private key if your laptop is stolen.

SSH Key File Locations

After running `ssh-keygen`:

```
ls -la ~/.ssh/
-rw-------  1 anas anas   464 Jan 30 10:00 id_ed25519       # PRIVATE
-rw-r--r--  1 anas anas    96 Jan 30 10:00 id_ed25519.pub   # Public
-rw-------  1 anas anas   200 Jan 30 10:00 authorized_keys  # Keys allowed in
-rw-r--r--  1 anas anas   444 Jan 30 10:00 known_hosts      # Servers trusted
```

Critical permissions:

File	Permission	Who Reads	Purpose
`id_ed25519`	`600`	Only you	Your identity (NEVER share!)
`id_ed25519.pub`	`644`	Anyone	Give this to servers

ssh-copy-id — Installing Keys on Server

What it does: Copies your public key to server's `authorized_keys` file.

Syntax: `ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server`

```
# Copy your public key to the server
ssh-copy-id -i ~/.ssh/id_ed25519.pub anas@192.168.1.50

# You'll be asked for your PASSWORD (one last time)
anas@192.168.1.50's password: ********

Number of key(s) added: 1

# Now test — no password needed!
ssh anas@192.168.1.50
anas@tuwaiq:~$    # You're in!
```

Manual Key Installation

If `ssh-copy-id` isn't available:

```
# On the SERVER — create .ssh directory
mkdir -p ~/.ssh
chmod 700 ~/.ssh

# Paste your public key
echo "ssh-ed25519 AAAA...your-key... anas@tuwaiq" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

Critical permissions (if wrong, SSH won't work!):

Path	Permission	Command
`~/.ssh/`	`700`	`chmod 700 ~/.ssh`
`~/.ssh/authorized_keys`	`600`	`chmod 600 ~/.ssh/authorized_keys`

~/.ssh/config — SSH Shortcuts

Tired of typing long SSH commands? Create aliases!

```
# Edit (or create) SSH config
nano ~/.ssh/config
```

Example config:

```
# Production web server
Host prod
    HostName 203.0.113.50
    User deploy
    IdentityFile ~/.ssh/id_ed25519
    Port 22

# Development server
Host dev
    HostName 192.168.1.100
    User anas
```

Now instead of: `ssh -i ~/.ssh/id_ed25519 deploy@203.0.113.50`

Just type: `ssh prod`

sshd_config — Securing the SSH Server

File: `/etc/ssh/sshd_config`

```
sudo nano /etc/ssh/sshd_config
```

Key security settings:

```
# Disable password authentication (keys only!)
PasswordAuthentication no

# Enable public key authentication
PubkeyAuthentication yes

# Disable root login
PermitRootLogin no

# Only allow specific users
AllowUsers anas deploy
```

After changing:

```
# Test config syntax first!
sudo sshd -t

# Then restart SSH (covered in M11)
sudo systemctl restart sshd
```

CRITICAL: Test Before Locking Yourself Out

Before disabling password authentication:

1. **Test key login in NEW terminal** (keep current session open)
2. **Verify it works without password**
3. **Only then** disable password authentication
4. **Keep one session open** until verified

> **If you close your session before testing, and keys don't work, you're locked out.**

Recovery if locked out:

- Cloud console access
- Serial console
- Rebuild instance

Common SSH Errors & Fixes

```
Permission denied (publickey).
```

Cause	Fix
Key not in authorized_keys	`ssh-copy-id` again
Wrong permissions	`chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys`
Wrong key file	Specify with `ssh -i /path/to/key`

More SSH Errors & Fixes

```
ssh: connect to host 192.168.1.50 port 22: Connection refused
```

Cause	Fix
SSH service not running	`sudo systemctl start sshd` (covered in M11)
Firewall blocking	`sudo ufw allow 22`
Wrong port	`ssh -p 2222 user@host`

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
```

Cause	Fix
Server reinstalled	`ssh-keygen -R hostname`
Possible attack	Verify with admin!

hostname / hostnamectl — Server Identity

What it does: View or change server's hostname.

```
# View current hostname
hostname
tuwaiq

# View detailed info
hostnamectl
   Static hostname: tuwaiq
   Operating System: Ubuntu 22.04.3 LTS
   Kernel: Linux 5.15.0-91-generic

# Change hostname (permanent)
sudo hostnamectl set-hostname tuwaiq-prod
```

> **Cloud tip:** In AWS/GCP, hostname is often set from instance metadata.

Aha! Moment 💡

Think about the security model:

GitHub
Only SSH keys or Personal Access Tokens (no passwords anymore)

AWS EC2
Launches with .pem key file, password SSH disabled by default

Google Cloud
SSH keys managed through metadata service

GitLab CI/CD
Deploy keys (SSH) for repository access

Terraform
SSH private keys for provisioning

Why this matters:

```
Password authentication:
- 10,000 brute force attempts per day
- Weak passwords = instant breach
- Shared passwords = no audit trail
- One leak = all servers compromised

SSH Key authentication:
- Brute force = mathematically impossible
- Private key never leaves your machine
- Each person has their own key
- Revoke one key = others unaffected
```

Real-world pattern:

- Day 1: Launch server with your SSH key
- Day 2: Add team keys to `authorized_keys`
- Day 3: Disable password auth
- Forever: Sleep soundly knowing bots can't get in

> SSH keys aren't "advanced security" — they're **the baseline**. Every production server on earth uses them.

SSH Keys & Security Cheat Sheet

Generating SSH keys:

```
# Ed25519 (recommended - modern, fast, secure)
ssh-keygen -t ed25519 -C "your-email@example.com"

# RSA 4096 (wider compatibility)
ssh-keygen -t rsa -b 4096 -C "your-email@example.com"

# Accept defaults or customize:
# - File: ~/.ssh/id_ed25519 (default is fine)
# - Passphrase: ALWAYS add one (protects if laptop stolen)
```

Installing keys on server:

```
# Automatic (easiest)
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server

# Manual method (if ssh-copy-id unavailable)
# On server:
mkdir -p ~/.ssh
chmod 700 ~/.ssh
echo "ssh-ed25519 AAAA...your-key..." >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

SSH client config (~/.ssh/config):

```
# Create shortcuts
nano ~/.ssh/config

# Example configuration:
Host prod
    HostName 203.0.113.50
    User deploy
    IdentityFile ~/.ssh/id_ed25519
    Port 22

Host dev
    HostName 192.168.1.100
    User anas
    ForwardAgent yes

# Now just: ssh prod
```

SSH server config (/etc/ssh/sshd_config):

```
# Edit config
sudo nano /etc/ssh/sshd_config

# Secure settings:
PasswordAuthentication no          # Disable passwords
PubkeyAuthentication yes           # Enable keys
PermitRootLogin no                 # No root login
AllowUsers anas deploy             # Whitelist users
Port 22                            # Change to hide from scans
ClientAliveInterval 300            # Keep connections alive
MaxAuthTries 3                     # Limit auth attempts

# Test and restart:
sudo sshd -t                       # Validate syntax
sudo systemctl restart sshd        # Apply changes (covered in M11)
```

Connecting with SSH:

```
# Basic connection
ssh user@hostname

# With specific key
ssh -i ~/.ssh/custom_key user@hostname

# With custom port
ssh -p 2222 user@hostname

# Run single command
ssh user@hostname 'sudo systemctl status nginx'

# Copy files (scp)
scp file.txt user@server:/path/to/dest
scp -r directory/ user@server:/path/

# Sync directories (rsync over SSH)
rsync -avz /local/dir/ user@server:/remote/dir/
```

SSH key management:

```
# List keys in SSH agent
ssh-add -l

# Add key to agent
ssh-add ~/.ssh/id_ed25519

# Remove all keys from agent
ssh-add -D

# View public key
cat ~/.ssh/id_ed25519.pub

# Get fingerprint
ssh-keygen -lf ~/.ssh/id_ed25519.pub
```

Mission Complete: SSH Keys and Security ✅

You locked down remote access:

01

Understood real password attack statistics

02

Generated SSH key pairs with ssh-keygen

03

Installed keys with ssh-copy-id

04

Disabled password authentication safely

05

Created SSH config shortcuts

06

Troubleshot common SSH errors

Commands you now know: `ssh-keygen`, `ssh-copy-id`, `ssh`, `hostname`, `hostnamectl`

> **Next mission:** Your server is secure. Your team can connect. But it's empty. No web server, no database, no monitoring tools. How do you install the software your application needs? And how do you keep it updated with security patches? Enter APT — the package manager that powers millions of Ubuntu servers.

Questions?